



COLEÇÃO ACADÊMICA · DATAGEO ACADEMY

Machine Learning Territorial com Scikit-Learn

Inteligência Artificial e Data Science

12 capítulos próprios · Código executável
Tabelas · Exercícios comentados

Autoria institucional: DataGeo Academy

Conhecimento que gera decisões | Edição 3.0 | 2026

Folha de rosto e controle documental

Machine Learning Territorial com Scikit-Learn

Autoria e edição institucional: DataGeo Academy.

CAMPO	REGISTRO
Edição	3.0 / 2026 / PDF pesquisável / 42 páginas
Organização	12 capítulos exclusivos deste tema; teoria, programação e aplicação
Dados	Sintéticos e identificados; não representam medições de um município
Bibliotecas	numpy>=2,<3, scikit-learn>=1.7,<2, pandas>=2.2,<4
Fontes editoriais	scripts_automacao/livros_ia; fonte completa anexada ao PDF
Verificação	12 laboratórios executados; asserções e saídas incorporadas
Caso integrador	Priorizar vistorias de unidades territoriais com regressão logística e floresta aleatória. Organize validação por blocos, escolha limiar segundo custo de falso negativo e orçamento, avalie o teste uma única vez e entregue pipeline, métricas por bloco e análise dos casos de erro.

O conteúdo deste volume foi escrito para sua sequência temática. A autoria é institucional; não há atribuição a pesquisadores ou revisores externos. A composição usa a identidade visual da Academy, mas os capítulos, exemplos e exercícios são independentes dos outros três volumes desta edição.

Os códigos são completos e executáveis em ambiente com as dependências indicadas. Os exercícios pedem alterações adicionais, e seus comentários orientam a solução. A validação automatizada verifica execução e propriedades declaradas; não constitui revisão científica externa nem certificação de resultados de campo.

REVISÃO	ESCOPO
3.0	Substituição do volume provisório por conteúdo específico e APIs reais.
Adaptação	Registrar autor da mudança, fonte, período, parâmetros e evidências.

Sumário técnico progressivo

CAP.	TEMA	PÁGINA
1	Problema supervisionado e mecanismo de geração	5
2	Blocos territoriais e separação independente	8
3	Pipeline, imputação e padronização sem vazamento	11
4	Atributos mistos e esquema de inferência	14
5	Baseline e regressão logística regularizada	17
6	Árvores, partições e controle de complexidade	20
7	Florestas, diversidade e importância de atributos	23
8	Desbalanceamento e métricas de detecção	26
9	Busca de hiperparâmetros com grupos	29
10	Calibração e confiabilidade das probabilidades	32
11	Limiar, orçamento e custo dos erros	35
12	Permutação, transferência e entrega do modelo	38

Cada capítulo possui três partes: fundamentos e uma tabela de decisões; laboratório integral com resultado calculado; exercícios autorais com comentários e uma aplicação ampliada. Os marcadores do PDF levam diretamente ao início de cada capítulo.

Os anexos incluem doze scripts Python independentes, fonte editorial em JSON, saídas de referência e requisitos do ambiente. Use um leitor com suporte a anexos ou o extrator fornecido no projeto. Execute os exemplos após ler o contrato e conferir as dependências.

COMPLEMENTO	PÁGINA
Introdução e percurso de estudo	4
Referências e ambiente	41
Projeto integrador e rubrica	42

Introdução e percurso de estudo

Uma boa pontuação de classificação não garante um bom modelo territorial. A posição das amostras, o momento da medição e a forma de construir o alvo podem produzir vazamento de informação. Este livro trata o aprendizado supervisionado como um experimento: definir a decisão, separar dados independentes, comparar baselines, escolher hiperparâmetros e analisar os erros que permanecem.

O caso integrador é a classificação de unidades territoriais para priorização de vistoria. As variáveis sintéticas representam relevo, acessibilidade e intensidade de ocupação. Não há cadastro real, diagnóstico ambiental nem previsão operacional. Os conjuntos são gerados por mecanismos declarados e as bibliotecas executam de fato treinamento, transformação, validação e avaliação.

O percurso começa pela definição de X e y , passa pela separação por grupos espaciais, pipelines e regressão logística, e avança por árvores, florestas, desbalanceamento, calibração e interpretação. O teste final permanece separado enquanto o modelo é escolhido. Os capítulos de métricas e limiar distinguem qualidade das probabilidades de utilidade da decisão tomada a partir delas.

O leitor deve conhecer arrays, tabelas e estatística básica. Os códigos usam Scikit-Learn 1.7 ou versão compatível. Cada laboratório funciona sem depender de estado de outro capítulo e contém uma asserção. As pequenas amostras reduzem tempo de execução e permitem conferir mecanismos; resultados de um conjunto sintético não estimam desempenho em cidades reais.

Ao final, o estudante será capaz de entregar um pipeline completo, um protocolo de validação por blocos, uma tabela de métricas, uma justificativa de limiar e um registro de limitações. Nenhuma variável deve entrar no modelo só porque melhora uma métrica: disponibilidade no momento da decisão, suporte espacial e plausibilidade precisam fazer parte da seleção.

CAPÍTULO 01 · TEORIA E MÉTODO

1. Problema supervisionado e mecanismo de geração

1.1 Fundamentos

Uma tarefa de classificação começa pelo significado do alvo. No caso territorial, y pode indicar ocorrência observada de um evento durante um período, enquanto X descreve variáveis disponíveis antes desse período. Um cadastro preenchido depois da vistoria pode revelar o resultado e não estar disponível na decisão. Esse vazamento torna o experimento diferente do uso pretendido.

No laboratório, a probabilidade de classe positiva depende de duas covariáveis por uma função logística. O sorteio Bernoulli transforma essa probabilidade em observação. Assim, o alvo não é uma regra determinística perfeita e existe erro irreduzível. A semente fixa torna o experimento repetível, mas não transforma a realização sorteada na verdade de um território.

A forma de X é n_{amostras} por $n_{\text{variáveis}}$; y possui uma entrada por amostra. Documente nomes, unidades, intervalos e origem. Um conjunto sintético controlado permite verificar se o modelo responde ao sinal esperado. Antes de qualquer ajuste, confira finitude, alinhamento e presença das duas classes. O treinamento sem um alvo bem definido apenas otimiza uma representação ambígua.

Hipóteses e aprofundamento técnico

Para um alvo Bernoulli, $E[y|X]=p(X)$ e $\text{Var}[y|X]=p(X)*(1-p(X))$. Mesmo um modelo que conheça exatamente p não acerta todos os sorteios quando a probabilidade está longe de zero e um. O erro irreduzível explica por que uma acurácia de treino perfeita pode representar memorização ou vazamento. A relação logística usa $\eta=\beta_0+\sum(\beta_j X_j)$; o sinal de β_j altera a razão de chances, condicionado às outras variáveis. Essa descrição é associativa. Em territórios, confusão entre definição de evento e registro do evento pode introduzir viés de detecção na própria construção de y .

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
X	400×2	Covariáveis disponíveis antes da decisão
y	400 classes	Ocorrência no horizonte declarado
p	Probabilidade latente	Não é o alvo observado
Semente	11	Reproduz o sorteio didático

Confira alinhamento entre linhas de X e y , definição do horizonte e disponibilidade de cada atributo. Não reporte dados sintéticos como evidência operacional.

CAPÍTULO 01 · PROGRAMAÇÃO

1.2 Laboratório executável

Arquivo incorporado: capitulo_01.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
rng = np.random.default_rng(11)
X = rng.normal(size=(400,2))
eta = 1.4*X[:,0] - 0.8*X[:,1] - 0.3
p = 1/(1+np.exp(-eta))
y = rng.binomial(1,p)
assert X.shape==(400,2) and y.shape==(400,)
assert np.isfinite(X).all() and set(y)=={0,1}
RESULTADO = {'amostras':len(y), 'positivos':int(y.sum()),
              'prevalencia':round(float(y.mean()),3)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"amostras": 400, "positivos": 186, "prevalencia": 0.465}
```

Leitura do código

default_rng controla a sequência de covariáveis e dos sorteios de classe. A probabilidade p permanece separada de y para que o estudante possa inspecionar ruído de observação. O teste set(y)=={0,1} garante duas classes nessa realização, não em todo tamanho de amostra. Nenhum modelo foi treinado ainda; primeiro se verifica o contrato do problema.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 01 · EXERCÍCIOS AUTORAIS

1.3 Caderno de aplicação

Exercício A · Modificação controlada

Acrescente uma terceira coluna igual a y com pequeno ruído. Treine um classificador com e sem essa coluna e explique por que o melhor resultado aparente representa um problema de disponibilidade temporal. Redija uma ficha de cada variável com data de medição e origem.

Comentário de solução

A coluna construída a partir de y contém informação posterior sobre o resultado. Mesmo com ruído, tende a aumentar a pontuação e viola o cenário de predição. Exclua-a por critério causal e temporal antes de dividir a amostra; não espere a validação detectar um vazamento presente em todos os grupos. A prevalência do laboratório é calculada da realização sorteada e pode mudar com outra semente.

Exercício B · Aplicação ampliada

Substitua o termo linear por uma interação $X_0 \times X_1$ e compare a interpretação do mecanismo. Mantenha separado o mecanismo conhecido do simulador e os parâmetros estimados pelo modelo; em dados reais, o mecanismo normalmente não é observável.

Critérios específicos de revisão

Confira alinhamento entre linhas de X e y, definição do horizonte e disponibilidade de cada atributo. Não reporte dados sintéticos como evidência operacional.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

2. Blocos territoriais e separação independente

2.1 Fundamentos

A divisão aleatória de pontos próximos pode colocar vizinhos quase idênticos em treino e teste. O modelo aprende características compartilhadas e a métrica fica otimista para regiões novas. Grupos espaciais representam unidades que não devem atravessar a fronteira de uma divisão. O tamanho dos blocos precisa dialogar com a escala de dependência e a finalidade da transferência.

GroupShuffleSplit separa grupos inteiros, mas não oferece estratificação automática das classes. Examine prevalência, tamanho e cobertura em cada divisão. Se um conjunto de teste contém somente uma classe, certas métricas ficam indefinidas. Coordenadas podem ajudar a construir blocos, porém fornecê-las como covariáveis também altera o que o modelo pode memorizar.

O exemplo reserva dois entre oito blocos sintéticos. A asserção de disjunção é mais importante que uma proporção exata de linhas: os grupos podem ter tamanhos diferentes. Documente o mapa dos blocos, a semente e o critério de alocação. O teste responde à generalização para grupos retidos, não automaticamente para outra cidade ou outro período.

Hipóteses e aprofundamento técnico

A independência relevante depende do uso futuro. Se a aplicação prevê um município novo, reter pequenos blocos dentro do mesmo município pode ser insuficiente. Se prevê um período novo, uma separação apenas espacial não testa o horizonte. Um buffer entre treino e teste pode reduzir compartilhamento de contexto em atributos extraídos de vizinhança. Grupos devem ser definidos antes de ver a pontuação, e seu mapa deve acompanhar o experimento. Repetir divisões fornece sensibilidade ao desenho, mas dobras correlacionadas não constituem automaticamente amostras independentes para calcular um intervalo estatístico.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Aleatório por linha	Vizinhos podem compartilhar divisão	Teste de interpolação
Por grupo	Blocos inteiros separados	Teste de transferência espacial
Temporal	Passado antes do futuro	Teste de horizonte
Inspeção	Classes e cobertura	Evita divisão inviável

Registre grupos exclusivos de cada divisão e inspecione distribuição de classes. Nenhum grupo de teste pode aparecer no ajuste.

CAPÍTULO 02 · PROGRAMAÇÃO

2.2 Laboratório executável

Arquivo incorporado: capitulo_02.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
from sklearn.model_selection import GroupShuffleSplit
rng = np.random.default_rng(12)
X = rng.normal(size=(160,3))
y = (X[:,0]+rng.normal(size=160)>0).astype(int)
grupos = np.repeat(np.arange(8),20)
split = GroupShuffleSplit(n_splits=1,test_size=.25,random_state=12)
treino, teste = next(split.split(X,y,grupos))
a, b = set(grupos[treino]), set(grupos[teste])
assert a.isdisjoint(b) and len(b)==2
RESULTADO = {'blocos_treino':sorted(map(int,a)),
              'blocos_teste':sorted(map(int,b)),
              'n_teste':len(teste)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"blocos_teste": [4, 6], "blocos_treino": [0, 1, 2, 3, 5, 7], "n_teste": 40}
```

Leitura do código

Os IDs de bloco têm vinte observações cada. O splitter retorna posições, usadas para selecionar X, y e grupos de forma alinhada. A disjunção é testada diretamente com conjuntos. Essa defesa detecta cruzamento de grupos mesmo se o número total de linhas parecer correto. Ordenar as linhas não substitui um splitter por grupos.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

2.3 Caderno de aplicação

Exercício A · Modificação controlada

Mude os tamanhos dos blocos para valores diferentes e confirme que 25% dos grupos não implica 25% das linhas. Depois crie um bloco com somente positivos e conte classes em cada divisão. Proponha uma estratégia de desenho amostral que permita avaliação consistente.

Comentário de solução

GroupShuffleSplit usa proporção de grupos. O laboratório retém 40 linhas porque todos os blocos têm 20, mas essa coincidência desaparece com tamanhos desiguais. A presença de duas classes precisa ser verificada explicitamente; não altere o teste repetidamente até obter uma pontuação favorável. Uma divisão planejada deve equilibrar cobertura suficiente com independência espacial.

Exercício B · Aplicação ampliada

Construa blocos a partir de coordenadas por floor(x/lado), floor(y/lado) e mantenha os IDs no manifesto. Compare dois tamanhos de bloco como análise de sensibilidade, sem escolher o menor somente por maximizar a métrica.

Critérios específicos de revisão

Registre grupos exclusivos de cada divisão e inspecione distribuição de classes. Nenhum grupo de teste pode aparecer no ajuste.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 03 · TEORIA E MÉTODO

3. Pipeline, imputação e padronização sem vazamento

3.1 Fundamentos

Transformações estimam parâmetros: a mediana da imputação e a média da padronização são informações aprendidas. Calculá-las antes da divisão permite que o teste influencie o treino. Um Pipeline ajusta as etapas apenas no conjunto recebido por fit e aplica as mesmas transformações em predict. Na validação cruzada, cada dobra recebe sua própria estimativa.

SimpleImputer com mediana trata ausência numérica, mas não explica sua causa. Ausência pode sinalizar falha de sensor ou cobertura seletiva. StandardScaler centra e escala atributos, útil para modelos lineares regularizados; árvores não precisam da mesma padronização. Dados categóricos demandam codificação própria, normalmente por ColumnTransformer e OneHotEncoder, sem converter categorias arbitrariamente em grandezas ordinais.

O laboratório injeta NaN em uma variável e treina uma regressão logística dentro do pipeline. A asserção confirma probabilidade finita, não a adequação da imputação. Consulte os parâmetros aprendidos e registre a ordem das etapas. Exportar apenas o classificador perderia as transformações e produziria previsões incompatíveis.

Hipóteses e aprofundamento técnico

A padronização implementa $z_j=(x_j-\mu_j)/\sigma_j$ com parâmetros aprendidos no treino. A imputação anterior muda quais valores participam desse cálculo, portanto a ordem das etapas é substantiva. Imputar mediana depois de escalar pode não produzir o mesmo comportamento em ausências. Na validação, a transformação deve ser clonada e ajustada em cada dobra. Se seleção de atributos ocorre fora do pipeline usando y completo, ainda há vazamento apesar de imputação e escala estarem corretas. Construa todo passo aprendido dentro do procedimento de ajuste e preserve as colunas necessárias à inferência.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Imputação	Mediana do treino	Não usar todo o conjunto
Escala	Média e desvio do treino	Reutilizar no teste
Classificador	LogisticRegression	Recebe matriz transformada
Persistência	Pipeline inteiro	Conserva ordem e parâmetros

Ajuste somente no treino, conserve o pipeline completo e confira nomes e ordem dos campos na inferência.

CAPÍTULO 03 · PROGRAMAÇÃO

3.2 Laboratório executável

Arquivo incorporado: capitulo_03.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
rng = np.random.default_rng(13)
X = rng.normal(size=(100,2)); y = (X[:,0]>0).astype(int)
X[:,9,1] = np.nan
modelo = make_pipeline(SimpleImputer(strategy='median'),
                       StandardScaler(), LogisticRegression(max_iter=300))
modelo.fit(X[:80],y[:80])
p = modelo.predict_proba(X[80:][:,1])
assert np.isfinite(p).all() and ((p>=0)&(p<=1)).all()
RESULTADO = {'n_predicoes':len(p),
             'medianas':modelo.named_steps['simpleimputer'].statistics_.tolist()}
print(RESULTADO)
```

Resultado de referência calculado

```
{"medianas": [0.18168872974237626, 0.06963722766094482], "n_predicoes": 20}
```

Leitura do código

$X[:80]$ contém treino e $X[80:]$ contém teste neste exercício de mecanismo. A ausência está apenas na segunda coluna. `named_steps` permite inspecionar estatísticas do imputador já ajustado. As probabilidades são verificadas quanto a finitude e domínio; isso não mede acerto nem prova que a imputação é adequada ao fenômeno.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

3.3 Caderno de aplicação

Exercício A · Modificação controlada

Desloque `X[80:,1]` em `+100` e compare a mediana estimada pelo pipeline com a mediana de toda a matriz. Demonstre que a mediana do treino permanece igual. Insira uma coluna categórica e monte um `ColumnTransformer` sem alterar o código da avaliação.

Comentário de solução

Os atributos do teste não participam de fit, portanto a mediana e a média aprendidas permanecem as mesmas após modificar apenas o teste. Estimar parâmetros com a matriz inteira altera a imputação e constitui vazamento. Para categoria nova, configure `OneHotEncoder(handle_unknown="ignore")` e declare o significado de um vetor de zeros para valores não vistos.

Exercício B · Aplicação ampliada

Avalie um indicador de ausência com `add_indicator=True` e compare resultados em validação por grupos. Se a ausência estiver ligada à região, o indicador pode funcionar no treino e falhar na transferência; examine o mecanismo em vez de assumir que mais colunas sempre ajudam.

Critérios específicos de revisão

Ajuste somente no treino, conserve o pipeline completo e confira nomes e ordem dos campos na inferência.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 04 · TEORIA E MÉTODO

4. Atributos mistos e esquema de inferência

4.1 Fundamentos

Bases territoriais combinam medidas numéricas e categorias, como declividade, distância à via e uso do solo. Codificar categorias por inteiros cria uma ordem e distância artificiais quando a classe não é ordinal. `OneHotEncoder` produz indicadores por categoria; `ColumnTransformer` aplica transformações diferentes a grupos de colunas, conservando um fluxo único de treinamento.

A seleção por nome reduz o risco de trocar colunas, mas o contrato ainda precisa verificar campos ausentes e categorias novas. `handle_unknown="ignore"` impede erro de execução diante de categoria não vista e produz zeros para seus indicadores; isso não significa que o modelo compreenda a nova classe. A decisão de aceitar, rejeitar ou sinalizar categorias desconhecidas pertence à política de inferência.

O laboratório combina uma variável numérica padronizada com uso do solo codificado. A transformação é ajustada só nas oito linhas de treino. Duas amostras de teste incluem uma categoria desconhecida para observar o comportamento definido. Nomes de atributos expandidos ajudam a inspecionar a matriz transformada. Não fixe manualmente a ordem das dummies em um script separado: exporte o pipeline completo.

Hipóteses e aprofundamento técnico

One-hot transforma k categorias em k indicadores por padrão. Com intercepto e regularização, a redundância pode ser administrada; escolher drop altera parametrização e interpretação. Categorias raras podem formar coeficientes instáveis e devem ser analisadas por suporte. `ColumnTransformer` pode produzir matriz esparsa dependendo dos componentes; não assuma ndarray denso em um exportador. `get_feature_names_out` documenta a ordem real das colunas após transformação. No uso territorial, categorias novas podem sinalizar mudança de legenda ou domínio, e merecem diagnóstico separado de uma predição numericamente válida.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Numérico	StandardScaler	Escala aprendida no treino
Nominal	OneHotEncoder	Não inventa ordem entre categorias
Desconhecido	Indicadores zerados	Sinalizar falta de suporte
Esquema	Seleção por nome	Conservar nomes e contrato

Conserve nomes, transformação e categorias no pipeline. Sinalize observações fora do domínio e verifique campos obrigatórios antes da inferência.

CAPÍTULO 04 · PROGRAMAÇÃO

4.2 Laboratório executável

Arquivo incorporado: capitulo_04.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
X=pd.DataFrame({'distancia':[10,20,30,40,80,90,100,110],
                'uso':['R','R','C','R','C','I','I','I']})
y=np.array([1,1,1,1,0,0,0,0])
prep=ColumnTransformer([('num',StandardScaler(),['distancia']),
                        ('cat',OneHotEncoder(handle_unknown='ignore'),['uso'])])
m=Pipeline([('prep',prep),('modelo',LogisticRegression())]).fit(X,y)
teste=pd.DataFrame({'distancia':[15,120], 'uso':['R','novo']})
p=m.predict_proba(teste)[:,-1]
assert np.isfinite(p).all() and len(p)==2
RESULTADO={'atributos':prep.get_feature_names_out().tolist(),
           'probabilidades':p.round(4).tolist()}
print(RESULTADO)
```

Resultado de referência calculado

```
{"atributos": ["num__distancia", "cat__uso_C", "cat__uso_I", "cat__uso_R"],
"probabilidades": [0.8822, 0.1119]}
```

Leitura do código

O pré-processador é parte do mesmo Pipeline que o classificador. distâncias são padronizadas e uso recebe indicadores C, I e R aprendidos no treino. A categoria novo não cria uma coluna durante predict. A saída permanece com duas probabilidades, mas a segunda observação precisa de sinalização de domínio porque sua categoria não foi ajustada.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

4.3 Caderno de aplicação

Exercício A · Modificação controlada

Troque `handle_unknown` para `error` e execute a predição da categoria novo. Depois crie uma rotina que conte categorias não vistas antes de `predict` e inclua esse diagnóstico junto às probabilidades. Acrescente uma coluna extra e confira o comportamento de `remainder`.

Comentário de solução

Com `handle_unknown="error"`, a transformação rejeita a categoria nova. Com `ignore`, o código executa, mas o vetor categórico é zero e a contribuição da categoria não foi aprendida. `ColumnTransformer` usa `remainder="drop"` por padrão, portanto uma coluna extra pode ser ignorada silenciosamente. Um contrato explícito deve distinguir campos permitidos, obrigatórios e proibidos, mesmo quando a API aceita a tabela.

Exercício B · Aplicação ampliada

Inclua uma categoria ordinal com uma ordem substantiva conhecida e compare `OrdinalEncoder` com `one-hot`. Documente a ordem; não use uma ordenação alfabética como substituto de uma escala temática. Mantenha a mesma divisão espacial das avaliações anteriores.

Critérios específicos de revisão

Conserve nomes, transformação e categorias no pipeline. Sinalize observações fora do domínio e verifique campos obrigatórios antes da inferência.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 05 · TEORIA E MÉTODO

5. Baseline e regressão logística regularizada

5.1 Fundamentos

Um modelo precisa superar uma referência pertinente. DummyClassifier estabelece uma comparação simples usando distribuição ou classe mais frequente. A regressão logística estima probabilidade binária como sigmoide de uma combinação linear; seus coeficientes descrevem a mudança no logaritmo da razão de chances, condicionada às outras variáveis e ao pré-processamento.

A regularização limita magnitude dos coeficientes e reduz instabilidade em atributos correlacionados. Em Scikit-Learn, C é inverso da força de regularização: C menor significa penalização mais forte. Coeficientes padronizados facilitam comparação relativa, mas não provam causalidade. Uma variável pode estar associada ao alvo devido à amostragem, a uma variável omitida ou à localização.

O exemplo compara acurácia balanceada de baseline e pipeline logístico em uma amostra com sinal claro. A semente controla o sorteio; o teste é separado antes de fit. Mesmo que o modelo supere a referência, analise erros, calibração e desempenho por grupo antes de recomendar uso. Uma baseline temporal ou uma regra de prioridade existente pode ser mais relevante que a classe majoritária em aplicações reais.

Hipóteses e aprofundamento técnico

A regressão logística penalizada minimiza perda de classificação combinada a uma restrição sobre coeficientes. Padronização muda a escala da penalização entre atributos. Quando todos os atributos usam unidades muito diferentes sem tratamento, a mesma penalização numérica não representa a mesma restrição substantiva. Uma baseline útil deve refletir uma decisão possível sem o modelo; em vistorias pode ser prioridade por regra de risco já usada. Compare ambos no mesmo teste, com o mesmo orçamento e o mesmo universo de amostras. Superar DummyClassifier é condição mínima, não certificação de utilidade.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Dummy	Classe majoritária	Referência mínima
Logística	Fronteira linear	Interpretação condicional
C menor	Regularização maior	Reduz magnitude dos coeficientes
C maior	Regularização menor	Pode elevar instabilidade

Compare com baseline, ajuste hiperparâmetros fora do teste e interprete coeficientes na escala efetivamente usada.

CAPÍTULO 05 · PROGRAMAÇÃO

5.2 Laboratório executável

Arquivo incorporado: capitulo_05.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score
X,y = make_classification(n_samples=300,n_features=4,
    n_informative=3,n_redundant=0,class_sep=1.5,random_state=14)
a,b,c,d = train_test_split(X,y,test_size=.3,stratify=y,random_state=14)
base = DummyClassifier(strategy='most_frequent').fit(a,c)
m = make_pipeline(StandardScaler(),LogisticRegression(C=.5)).fit(a,c)
s0 = balanced_accuracy_score(d,base.predict(b))
s1 = balanced_accuracy_score(d,m.predict(b))
assert s1>s0
RESULTADO = {'baseline':round(s0,3),'logistica':round(s1,3)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"baseline": 0.5, "logistica": 0.956}
```

Leitura do código

`train_test_split` estratifica `y` para preservar classes no pequeno teste sintético. O classificador logístico está dentro de `make_pipeline`, então escala é aprendida só em `a`. `balanced_accuracy` calcula média de recall das classes e a classe constante tende a 0,5 no caso binário. A asserção exige melhoria nessa realização controlada.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

5.3 Caderno de aplicação

Exercício A · Modificação controlada

Teste $C=0,01$, 1 e 100 com a mesma divisão. Compare norma dos coeficientes e desempenho. Não escolha C pelo teste: use uma divisão de validação dentro do treino. Explique o efeito de acrescentar um atributo perfeitamente duplicado.

Comentário de solução

A penalização distribui peso entre atributos correlacionados; coeficientes individuais podem mudar sem grande alteração nas probabilidades. Valores maiores de C permitem pesos maiores, mas não garantem melhora na generalização. O teste deve permanecer reservado enquanto C é escolhido. A acurácia balanceada da classe constante é 0,5 quando as duas classes estão presentes.

Exercício B · Aplicação ampliada

Adicione uma regra territorial de referência baseada em um único atributo. Compare orçamento de vistorias, recall e precisão com a regressão logística. Uma melhoria estatística pequena pode ter impacto operacional grande ou pequeno conforme o custo de cada tipo de erro.

Critérios específicos de revisão

Compare com baseline, ajuste hiperparâmetros fora do teste e interprete coeficientes na escala efetivamente usada.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 06 · TEORIA E MÉTODO

6. Árvores, partições e controle de complexidade

6.1 Fundamentos

Uma árvore de decisão particiona o espaço de atributos por regras sucessivas. Pode capturar relações não lineares e interações sem padronização. Entretanto, uma árvore profunda consegue memorizar ruído e pequenas diferenças da amostra. `max_depth`, `min_samples_leaf` e poda controlam a complexidade, cada um restringindo um aspecto diferente da partição.

O número mínimo por folha evita probabilidades apoiadas em uma ou duas observações. Em dados territoriais, essas observações podem pertencer ao mesmo bloco, de modo que quantidade de linhas não equivale a informação independente. Um limite de profundidade facilita leitura, mas uma regra curta ainda pode refletir uma covariável com vazamento.

O laboratório usa um padrão de luas, deliberadamente não linear, e compara árvore rasa com profunda. O objetivo não é demonstrar que uma profundidade sempre vence, mas visualizar diferença entre capacidade de ajuste e desempenho retido. A taxa de treino deve ser lida junto à taxa do teste. Regras interpretáveis precisam ser verificadas quanto a suporte amostral, estabilidade e plausibilidade.

Hipóteses e aprofundamento técnico

A impureza Gini de uma folha é $1 - \sum(p_k^2)$. Um corte escolhe reduzir impureza ponderada das folhas resultantes. Isso otimiza homogeneidade no treino e não garante estabilidade geográfica. Árvores podem variar com pequenas mudanças de amostra porque um corte inicial modifica todos os seguintes. Limitar folhas ou profundidade restringe a complexidade; poda por custo-complexidade compara ganho de ajuste com tamanho da árvore. Para regras territoriais divulgadas, registre intervalo de atributo, quantidade de casos e distribuição de classe em cada folha, evitando extrapolar uma regra para regiões sem suporte.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
<code>max_depth</code>	Número de níveis	Limita sequência de regras
<code>min_samples_leaf</code>	Suporte por folha	Evita folhas frágeis
Treino	Capacidade de ajuste	Pode premiar memorização
Teste	Generalização retida	Não usar para seleção repetida

Examine diferença treino-teste, suporte das folhas e estabilidade das regras. Não escolha profundidade olhando apenas a pontuação de treino.

CAPÍTULO 06 · PROGRAMAÇÃO

6.2 Laboratório executável

Arquivo incorporado: capitulo_06.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
X, y = make_moons(n_samples=300, noise=.25, random_state=15)
a, b, c, d = train_test_split(X, y, test_size=.3, random_state=15, stratify=y)
raso = DecisionTreeClassifier(max_depth=3, min_samples_leaf=10,
                             random_state=15).fit(a, c)
fundo = DecisionTreeClassifier(random_state=15).fit(a, c)
assert fundo.get_depth() >= raso.get_depth()
RESULTADO = {'raso_treino': round(raso.score(a, c), 3),
              'raso_teste': round(raso.score(b, d), 3),
              'fundo_treino': round(fundo.score(a, c), 3),
              'fundo_teste': round(fundo.score(b, d), 3)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"fundo_teste": 0.933, "fundo_treino": 1.0, "raso_teste": 0.9, "raso_treino": 0.886}
```

Leitura do código

make_moons cria um padrão não linear com ruído, útil para comparar capacidade das árvores. As duas recebem o mesmo treino e random_state. get_depth verifica diferença estrutural e score calcula acurácia de treino e teste. A asserção não exige que a árvore rasa vença sempre; esse seria um compromisso indevido com uma realização específica.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

6.3 Caderno de aplicação

Exercício A · Modificação controlada

Use `export_text` para imprimir as regras da árvore rasa. Identifique a folha com menor número de amostras e registre sua distribuição de classes. Repita o ajuste em cinco reamostragens do treino e compare quais atributos aparecem na raiz.

Comentário de solução

A árvore profunda alcança ajuste muito alto no treino, mas isso não é garantia de melhor teste. Uma folha pequena fornece uma frequência de classe instável. A mudança de atributo na raiz entre reamostragens sinaliza instabilidade da explicação, mesmo quando a pontuação média muda pouco. `export_text` é uma representação das regras aprendidas, não uma explicação causal.

Exercício B · Aplicação ampliada

Monte uma curva de validação para profundidades 2, 3, 5 e 8 usando grupos territoriais. Mostre média e dispersão por dobra. Prefira uma configuração simples quando diferenças pequenas estiverem dentro da variação entre blocos.

Critérios específicos de revisão

Examine diferença treino-teste, suporte das folhas e estabilidade das regras. Não escolha profundidade olhando apenas a pontuação de treino.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 07 · TEORIA E MÉTODO

7. Florestas, diversidade e importância de atributos

7.1 Fundamentos

RandomForestClassifier combina árvores treinadas com amostras bootstrap e subconjuntos de atributos. A diversidade reduz variância em relação a uma árvore isolada, mas não corrige vazamento nem uma divisão inadequada. A floresta pode aprender padrões locais que não se transferem para novos territórios. n_estimators controla quantidade de árvores; min_samples_leaf ajuda a suavizar folhas.

A importância por redução de impureza pode favorecer atributos com muitos valores possíveis e não mede causalidade. OOB score usa observações fora do bootstrap de cada árvore, porém amostras vizinhas podem continuar em árvores treinadas; OOB não substitui validação espacial independente. Em estudos territoriais, mantenha o teste por blocos mesmo quando o algoritmo fornece avaliação interna.

O exemplo usa oito blocos e uma classe gerada por dois atributos. O teste retém dois blocos inteiros. Além de probabilidades finitas, o laboratório confere a soma das importâncias por impureza. Essa soma igual a um é uma propriedade de normalização, não prova de qualidade. Registre parâmetros e compare o modelo com um pipeline linear sob a mesma divisão.

Hipóteses e aprofundamento técnico

A redução de variância de um ensemble depende da correlação entre erros das árvores. Mais árvores muito semelhantes não proporcionam o mesmo benefício que árvores diversas. max_features restringe candidatos em cada corte e contribui para diversidade; min_samples_leaf controla suporte das probabilidades locais. Uma floresta ainda pode memorizar um identificador ou contexto espacial. OOB avalia linhas não usadas em cada bootstrap, mas outras linhas do mesmo grupo podem ter sido usadas. Por isso escolha validação segundo a pergunta de transferência, independentemente das conveniências internas do algoritmo.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Bootstrap	Amostras com reposição	Diversifica árvores
Subatributos	Competição parcial	Reduz correlação entre árvores
OOB	Retidos do bootstrap	Não garante separação territorial
Importância	Redução de impureza	Não implica causalidade

Conserve grupos independentes e declare qual medida de importância está sendo divulgada. Não apresente OOB como validação de transferência espacial.

CAPÍTULO 07 · PROGRAMAÇÃO

7.2 Laboratório executável

Arquivo incorporado: capitulo_07.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import balanced_accuracy_score
rng=np.random.default_rng(16)
X=rng.normal(size=(240,3)); grupos=np.repeat(np.arange(8),30)
y=(X[:,0]*X[:,1]+.3*X[:,2]>0).astype(int)
a,b=next(GroupShuffleSplit(n_splits=1,test_size=.25,
    random_state=16).split(X,y,grupos))
m=RandomForestClassifier(n_estimators=80,min_samples_leaf=3,
    random_state=16,n_jobs=1).fit(X[a],y[a])
p=m.predict_proba(X[b])[:,1]
assert np.isfinite(p).all()
assert abs(m.feature_importances_.sum()-1)<1e-8
RESULTADO={'balanced_accuracy':round(balanced_accuracy_score(y[b],p>=.5),3),
    'importancias_impureza':m.feature_importances_.round(3).tolist()}
print(RESULTADO)
```

Resultado de referência calculado

```
{"balanced_accuracy": 0.759, "importancias_impureza": [0.355, 0.315, 0.33]}
```

Leitura do código

A classe sintética contém interação $X_0 \times X_1$ e um termo de X_2 . Grupos retidos não entram no fit. `predict_proba` fornece a coluna da classe positiva e o limiar 0,5 cria decisão binária apenas para a métrica. Importâncias por impureza são normalizadas; sua soma um é verificação estrutural, não resultado causal.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 07 · EXERCÍCIOS AUTORAIS

7.3 Caderno de aplicação

Exercício A · Modificação controlada

Acrescente uma coluna identificadora aleatória quase única por amostra. Compare importância por impureza e importância por permutação no teste. Depois substitua a divisão por linha pela divisão por grupo e discuta qual pergunta cada avaliação responde.

Comentário de solução

Uma coluna com muitos valores oferece mais candidatos a cortes e pode receber importância por impureza sem contribuir à generalização. Permutar a coluna no teste mede perda de pontuação sob o modelo ajustado; se a contribuição for nula, a importância tende a ficar próxima de zero ou variar ao redor dela. Nenhuma medida deve ser usada isoladamente como seleção causal.

Exercício B · Aplicação ampliada

Compare folhas mínimas 1, 3 e 10 em validação por grupos. Avalie estabilidade das probabilidades e tempo de execução, mantendo as mesmas divisões. Aumentar árvores reduz variação do ensemble, mas não resolve variáveis inadequadas ou cobertura amostral insuficiente.

Critérios específicos de revisão

Conserve grupos independentes e declare qual medida de importância está sendo divulgada. Não apresente OOB como validação de transferência espacial.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

8. Desbalanceamento e métricas de detecção

8.1 Fundamentos

Quando eventos positivos são raros, acurácia pode ser alta para um modelo que nunca os detecta. Precision descreve a fração de alertas corretos; recall descreve a fração de ocorrências encontradas. A curva precisão-recall expõe o compromisso ao variar o limiar. Average precision resume a curva e depende da prevalência do conjunto avaliado.

`class_weight` altera a contribuição das classes na função de ajuste. Não cria novas observações nem garante probabilidades calibradas. Reamostragem também precisa ocorrer dentro do treino de cada dobra; equilibrar o conjunto antes da divisão pode duplicar informação no teste. A amostra de avaliação deve refletir a população de interesse ou declarar pesos e desenho amostral.

O exemplo treina logística ponderada em uma base com aproximadamente 10% de positivos. A matriz de confusão preserva a interpretação das contagens, enquanto AP resume o ranking. O limiar 0,5 é apenas uma configuração inicial. Em inspeções territoriais, o número de vistorias e o custo de perder uma ocorrência frequentemente são mais importantes que a acurácia global.

Hipóteses e aprofundamento técnico

Precisão e recall dependem de qual classe foi definida como positiva. Em uma população com 1% de ocorrência, mesmo uma pequena taxa de falso positivo pode gerar muitos alertas sem evento. O valor preditivo positivo combina sensibilidade, especificidade e prevalência; transferir um modelo para outra prevalência pode mudar a utilidade dos alertas. Curvas ROC e PR destacam aspectos diferentes. Escolha métrica segundo a decisão e apresente prevalência do teste. Não use reamostragem para equilibrar o teste e depois interprete precision como se representasse a população original.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Acurácia	Acertos / total	Pode ocultar classe rara
Precision	$TP / (TP+FP)$	Qualidade dos alertas
Recall	$TP / (TP+FN)$	Cobertura de ocorrências
AP	Resumo do ranking	Comparar sob prevalência declarada

Mantenha a classe rara no teste, apresente contagens junto às taxas e registre o limiar de cada resultado.

CAPÍTULO 08 · PROGRAMAÇÃO

8.2 Laboratório executável

Arquivo incorporado: capitulo_08.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, confusion_matrix
X,y=make_classification(n_samples=500,n_features=5,n_redundant=0,
                        weights=[.9,.1],random_state=17)
a,b,c,d=train_test_split(X,y,test_size=.3,stratify=y,random_state=17)
m=make_pipeline(StandardScaler(),LogisticRegression(
    class_weight='balanced',max_iter=300)).fit(a,c)
p=m.predict_proba(b)[:,-1]; cm=confusion_matrix(d,p>=.5,labels=[0,1])
assert cm.sum()==len(d)
RESULTADO={'AP':round(average_precision_score(d,p),3),
            'matriz_TN_FP_FN_TP':cm.tolist(),'prevalencia':round(float(d.mean()),3)}
print(RESULTADO)
```

Resultado de referência calculado

```
{ "AP": 0.977, "matriz_TN_FP_FN_TP": [[132, 3], [1, 14]], "prevalencia": 0.1 }
```

Leitura do código

weights define o desbalanceamento do simulador e class_weight="balanced" modifica o ajuste logístico. A matriz usa labels=[0,1], fixando ordem TN, FP, FN e TP. AP recebe probabilidades, não as classes geradas pelo limiar. cm.sum confere que todas as amostras do teste foram contabilizadas.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

8.3 Caderno de aplicação

Exercício A · Modificação controlada

Calcule precision e recall diretamente da matriz de confusão. Repita com limiar 0,2 e 0,8 e registre o número de alertas. Compare AP com a prevalência e explique por que mudar o limiar não muda AP das mesmas probabilidades.

Comentário de solução

TP, FP e FN variam quando o limiar muda. Ao reduzi-lo, normalmente há mais alertas e maior recall, com possível perda de precision. AP usa o ranking das probabilidades e não um único vetor binário; portanto permanece igual enquanto p não muda. Se o denominador de precision for zero, declare a convenção usada em vez de apresentar um valor sem contexto.

Exercício B · Aplicação ampliada

Construa uma tabela de orçamento com 10, 20 e 30 vistorias selecionando as maiores probabilidades. Conte ocorrências capturadas em cada orçamento. Não compare AP de regiões com prevalências diferentes sem considerar a diferença de dificuldade e amostragem.

Critérios específicos de revisão

Mantenha a classe rara no teste, apresente contagens junto às taxas e registre o limiar de cada resultado.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

9. Busca de hiperparâmetros com grupos

9.1 Fundamentos

Hiperparâmetros são escolhidos antes de ajustar o modelo final e podem ser otimizados por validação cruzada. Se cada dobra deve reter territórios inteiros, o splitter precisa receber grupos. Passar grupos a fit do GridSearchCV permite que GroupKFold forme as divisões. A escolha do estimador não deve usar o teste final.

Uma busca extensa aumenta custo e oportunidades de selecionar flutuações favoráveis. Defina uma grade pequena orientada por hipóteses e uma métrica coerente com a decisão. A pontuação média esconde diferenças entre blocos; examine as colunas split*_test_score e a dispersão. Dados com classes ausentes em uma dobra precisam de redesenho ou métricas compatíveis.

O laboratório seleciona C em uma regressão logística com padronização dentro do pipeline. Os seis grupos aparecem apenas na validação interna. O exemplo não inclui teste final para evitar confundir o resultado da busca com estimativa externa de desempenho. Depois de fixar o procedimento, ajuste no treino completo e avalie o conjunto reservado uma única vez.

Hipóteses e aprofundamento técnico

Uma seleção por máximo entre muitas configurações pode favorecer ruído de validação. Esse viés de seleção aumenta quando se consulta repetidamente a mesma base e se alteram variáveis, métricas ou divisões. Validação aninhada separa uma camada interna de escolha e uma externa de avaliação. Seu custo exige planejamento, não eliminação da independência. Registre a grade antes da execução, conserve resultados de todas as alternativas e relate dispersão por bloco. Se a melhor configuração muda muito entre dobras, o resultado pode indicar fragilidade amostral, e não necessidade de uma grade ainda maior.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Grid	C=0,1; 1; 10	Hipóteses limitadas
GroupKFold	Três dobras	Grupos não atravessam divisão
Pipeline	Escala em cada treino	Evita vazamento de parâmetros
Teste final	Fora da busca	Estimativa externa

Confirme grupos separados, transformação dentro do pipeline e teste fora da otimização. Não transforme dispersão entre dobras em intervalo de confiança independente sem justificativa.

CAPÍTULO 09 · PROGRAMAÇÃO

9.2 Laboratório executável

Arquivo incorporado: capitulo_09.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
from sklearn.model_selection import GridSearchCV, GroupKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
rng=np.random.default_rng(18)
X=rng.normal(size=(180,3)); grupos=np.repeat(np.arange(6),30)
y=(X[:,0]-.5*X[:,1]+rng.normal(size=180)*.5>0).astype(int)
pipe=make_pipeline(StandardScaler(),LogisticRegression(max_iter=300))
busca=GridSearchCV(pipe,{'logisticregression__C': [.1,1,10]},
    cv=GroupKFold(n_splits=3),scoring='balanced_accuracy',n_jobs=1)
busca.fit(X,y,groups=grupos)
assert busca.best_params_['logisticregression__C'] in [.1,1,10]
RESULTADO={'C':busca.best_params_['logisticregression__C'],
    'score_validacao':round(float(busca.best_score_),3)}
print(RESULTADO)
```

Resultado de referência calculado

```
{"C": 1, "score_validacao": 0.81}
```

Leitura do código

Os nomes com dois underscores endereçam o parâmetro C dentro da etapa logisticregression do pipeline. groups é repassado ao CV e não vira uma coluna de X. Cada candidata é ajustada em cada dobra com sua própria escala. best_score é um resultado de seleção interna e não uma pontuação externa do modelo escolhido.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

9.3 Caderno de aplicação

Exercício A · Modificação controlada

Imprima grupos de treino e validação de cada dobra e confira disjunção. Adicione `return_train_score=True` e compare diferenças treino-validação. Explique por que reportar `best_score_` como desempenho final é otimista.

Comentário de solução

`best_score_` é usado para selecionar entre alternativas e, portanto, participa da decisão de modelo. Uma estimativa independente exige um teste não consultado nessa escolha ou validação aninhada. Os grupos devem ser repassados ao splitter; omiti-los em `GroupKFold` impede a separação pretendida. O pipeline faz a padronização separadamente em cada dobra.

Exercício B · Aplicação ampliada

Implemente validação aninhada com uma dobra externa por grupos e uma busca interna. Registre parâmetros vencedores em cada divisão e compare estabilidade. Use poucos valores para manter custo controlado e não interpretar uma configuração instável como descoberta definitiva.

Critérios específicos de revisão

Confirme grupos separados, transformação dentro do pipeline e teste fora da otimização. Não transforme dispersão entre dobras em intervalo de confiança independente sem justificativa.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 10 · TEORIA E MÉTODO

10. Calibração e confiabilidade das probabilidades

10.1 Fundamentos

Discriminação e calibração são propriedades distintas. Um modelo pode ordenar corretamente os casos e ainda atribuir probabilidades exageradas. Brier score mede erro quadrático das probabilidades; curvas de confiabilidade comparam frequência observada e probabilidade média em faixas. Uma probabilidade de 0,8 só admite interpretação frequencial após avaliação compatível com a população.

CalibratedClassifierCV aprende uma transformação a partir de predições em dados retidos nas divisões internas. sigmoid costuma ser mais estável em amostras menores; isotonic é flexível e exige mais informação. Calibrar com o teste final contamina a avaliação. A estratégia de CV da calibração também precisa respeitar a independência espacial desejada.

O laboratório demonstra a API com divisões estratificadas em uma amostra independente sintética. Isso ensina o mecanismo e não constitui uma validação territorial: para um projeto espacial, forneça divisões por grupos previamente construídas. A asserção verifica a faixa das probabilidades e o intervalo matemático do Brier, sem prometer melhoria em toda realização.

Hipóteses e aprofundamento técnico

Para probabilidade calibrada, entre casos próximos a $p=0,7$ espera-se aproximadamente 70% de positivos em uma população comparável. Essa interpretação não exige perfeição em uma amostra pequena, mas requer evidência. Brier combina aspectos de calibração e discriminação e não deve ser descrito como medida exclusiva de calibração. Uma curva por faixas também depende de binning e suporte. Mudança de prevalência, amostragem de casos e class_weight podem alterar probabilidade. Preserve um conjunto de avaliação que represente a população, ou documento a correção e os pesos usados.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Ranking	Ordenação dos casos	Não garante probabilidade correta
Brier	Erro quadrático de p	Menor é melhor no mesmo teste
Sigmoid	Transformação paramétrica	Menor flexibilidade
Isotonic	Transformação monotônica	Precisa de mais observações

Nunca ajuste a calibradora no teste final; apresente curva, contagens por faixa e Brier sob a população declarada.

CAPÍTULO 10 · PROGRAMAÇÃO

10.2 Laboratório executável

Arquivo incorporado: capitulo_10.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import brier_score_loss
X,y=make_classification(n_samples=400,n_features=5,
                        n_redundant=0,random_state=19)
a,b,c,d=train_test_split(X,y,test_size=.25,stratify=y,random_state=19)
base=RandomForestClassifier(n_estimators=60,max_depth=4,
                           random_state=19,n_jobs=1)
m=CalibratedClassifierCV(base,method='sigmoid',cv=3).fit(a,c)
p=m.predict_proba(b)[:,-1]; brier=brier_score_loss(d,p)
assert 0<=brier<=1 and ((p>=0)&(p<=1)).all()
RESULTADO={'brier':round(float(brier),4),
           'prob_media':round(float(p.mean()),4),'prevalencia':float(d.mean())}
print(RESULTADO)
```

Resultado de referência calculado

```
{"brier": 0.0495, "prevalencia": 0.5, "prob_media": 0.5358}
```

Leitura do código

CalibratedClassifierCV faz divisões internas para ajustar a transformação sigmoid sobre saídas retidas. A floresta-base não é previamente ajustada no teste. Brier usa a probabilidade contínua e y observado. O exemplo não contém grupos, portanto sua divisão é apenas uma demonstração independente da API e não um protocolo espacial pronto.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

10.3 Caderno de aplicação

Exercício A · Modificação controlada

Compare Brier de uma floresta não calibrada com o modelo calibrado usando o mesmo teste. Calcule frequência observada e p média em cinco faixas. Repita com três seeds e explique por que a calibração não deve ser declarada vencedora apenas por uma realização.

Comentário de solução

Brier menor indica melhor erro probabilístico naquele conjunto, mas o ganho pode variar. Faixas com poucas observações têm frequência instável; inclua quantidade por faixa. A prevalência média próxima à probabilidade média é condição útil, mas não suficiente para calibração em todas as faixas. A divisão interna do exemplo é estratificada, não espacial, e deve ser substituída em dados dependentes.

Exercício B · Aplicação ampliada

Monte as divisões internas com GroupKFold no treino e passe a lista de pares de índices a cv. Verifique a presença das duas classes em cada subconjunto antes de calibrar. Reserve outro conjunto para avaliar calibração e uma posterior transferência temporal.

Critérios específicos de revisão

Nunca ajuste a calibradora no teste final; apresente curva, contagens por faixa e Brier sob a população declarada.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

CAPÍTULO 11 · TEORIA E MÉTODO

11. Limiar, orçamento e custo dos erros

11.1 Fundamentos

O limiar traduz uma probabilidade em ação. O valor 0,5 não é universalmente ótimo: falso negativo e falso positivo podem ter custos muito diferentes. Quando probabilidades estão calibradas e os custos são conhecidos, o limiar teórico depende desses custos. Com orçamento rígido, escolher os k maiores escores pode ser mais adequado que um limiar fixo.

Ajustar limiar é parte da seleção do procedimento e deve usar validação, não o teste final. Uma tabela de custos precisa declarar unidade, período e hipótese. Custos inventados para um exercício não representam avaliação econômica de um projeto. Além do custo agregado, analise quem recebe a intervenção e se existem restrições de acesso ou distribuição territorial.

O laboratório usa seis probabilidades e rótulos conhecidos para calcular custo com falso negativo valendo cinco e falso positivo valendo um. Compara três limiares sem treinar modelo. A vantagem é permitir conferir cada decisão manualmente. O custo do menor limiar neste exemplo deriva dos casos específicos e não autoriza aplicar o mesmo valor em qualquer base.

Hipóteses e aprofundamento técnico

Com probabilidades calibradas e dois custos constantes, agir pode ser preferível quando $p \cdot C_{FN}$ supera $(1-p) \cdot C_{FP}$, levando ao limiar $C_{FP}/(C_{FP}+C_{FN})$. Essa expressão pressupõe custos definidos e ausência de outras restrições. Orçamento limitado, capacidade de atendimento ou regra de equidade modifica o problema. Além disso, custo de FN pode depender da intensidade ou da região. Portanto um limiar obtido por uma fórmula não dispensa avaliação de finalidade e suporte. Use validação para escolher a política e teste para medir suas consequências após congelar a escolha.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
FP	Vistoria sem ocorrência	Custo didático 1
FN	Ocorrência não detectada	Custo didático 5
Limiar	Regra $p \geq t$	Escolher na validação
Top-k	Orçamento de ações	Não mantém custo fixo por probabilidade

Escolha regra fora do teste, mantenha custos e orçamento explícitos e divulgue quantidades de alertas junto às métricas.

CAPÍTULO 11 · PROGRAMAÇÃO

11.2 Laboratório executável

Arquivo incorporado: capitulo_11.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
p=np.array([.1,.3,.45,.55,.7,.9])
y=np.array([0,1,0,1,1,1])
res=[]
for t in [.25,.5,.75]:
    alertas=p>=t
    fp=int((y==0)&alertas).sum()
    fn=int((y==1)&~alertas).sum()
    res.append({'limiar':t,'FP':fp,'FN':fn,
               'custo':fp+5*fn,'alertas':int(alertas.sum())})
assert [r['custo'] for r in res]==[1,5,15]
RESULTADO=res
print(RESULTADO)
```

Resultado de referência calculado

```
[{"FN": 0, "FP": 1, "alertas": 5, "custo": 1, "limiar": 0.25}, {"FN": 1, "FP": 0, "alertas": 3, "custo": 5, "limiar": 0.5}, {"FN": 3, "FP": 0, "alertas": 1, "custo": 15, "limiar": 0.75}]
```

Leitura do código

O loop avalia três políticas sobre as mesmas probabilidades. FP e FN são contados por máscaras booleanas independentes; custo usa pesos declarados. A sequência [1,5,15] é conferível diretamente pelos seis casos. Não houve otimização de modelo: o exercício isola a camada de decisão, que pode variar sem retreinar os escores.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 11 · EXERCÍCIOS AUTORAIS

11.3 Caderno de aplicação

Exercício A · Modificação controlada

Imponha orçamento de três alertas e selecione os maiores p . Compare custo e recall com o limiar 0,25. Depois aumente custo de FP para 10 e recalcule a tabela. Mostre que a regra preferida depende da finalidade.

Comentário de solução

Top-3 seleciona $p=0,55$, 0,7 e 0,9, encontrando três dos quatro positivos: recall=0,75 e custo=5. O limiar 0,25 gera cinco alertas, encontra todos os positivos e custa 1, mas viola o orçamento de três. Com FP custando 10, esse limiar passa a custo 10; o limiar 0,5 permanece em 5. A avaliação precisa considerar simultaneamente restrição e custo.

Exercício B · Aplicação ampliada

Inclua uma regra de cobertura mínima por bloco territorial e selecione alertas com essa restrição. Compare perda de eficiência global e ganho de distribuição. Registre a decisão como política operacional, separada dos parâmetros de treinamento.

Critérios específicos de revisão

Escolha regra fora do teste, mantenha custos e orçamento explícitos e divulgue quantidades de alertas junto às métricas.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

12. Permutação, transferência e entrega do modelo

12.1 Fundamentos

A importância por permutação mede quanto uma métrica cai quando um atributo é embaralhado no conjunto avaliado. É uma medida dependente do modelo, da amostra e da métrica. Atributos correlacionados podem substituir uns aos outros, reduzindo a importância individual mesmo quando o conjunto contém sinal relevante. O método não identifica efeitos causais.

Uma análise de entrega inclui desempenho por território, matriz de confusão, calibração, limiar e condições de uso. Mudanças na distribuição das covariáveis ou na relação com o alvo podem exigir reavaliação. Monitorar apenas a distribuição de p não detecta todos os problemas. Rótulos futuros e revisão dos casos são necessários para medir desempenho após implantação.

No laboratório, o alvo depende principalmente de X_0 e a permutação permite verificar se o pipeline explora esse atributo. A avaliação usa um teste separado. Não há exigência de que toda importância seja positiva; resultados negativos pequenos podem aparecer por variação amostral. O pacote entregue deve manter o pipeline, esquema de entrada, versões e evidências do protocolo, sem carregar arquivos de modelo de origem não confiável.

Hipóteses e aprofundamento técnico

Permutação quebra a relação de uma coluna com y e também com outras covariáveis. Quando há correlação, pode criar combinações fora do domínio observado; a queda de métrica inclui esse efeito. Importância condicional é uma pergunta diferente e exige métodos próprios. A interpretação também depende do desempenho inicial: atribuir importância a um modelo ruim pode explicar ruído. Calcule no conjunto pertinente e apresente variação entre repetições. Transferência temporal pode alterar o conceito mesmo quando histogramas de X permanecem semelhantes; só monitorar covariáveis não basta para detectar esse caso.

Decisões e limites do procedimento

ELEMENTO	CONFIGURAÇÃO	INTERPRETAÇÃO
Permutação	Perda de pontuação	Depende do teste e da métrica
Correlação	Atributos substitutos	Importância individual pode diminuir
Transferência	Novo bloco ou período	Requer avaliação específica
Entrega	Pipeline e contrato	Conservar pré-processamento

Interprete importância como dependência preditiva, preserve o pipeline e monitore erros por território após qualquer transferência.

CAPÍTULO 12 · PROGRAMAÇÃO

12.2 Laboratório executável

Arquivo incorporado: capitulo_12.py. Execute em Python 3 com as dependências deste volume. O exemplo é independente: imports, entradas, processamento e verificação estão no bloco abaixo.

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
rng=np.random.default_rng(20)
X=rng.normal(size=(300,3)); y=(X[:,0]+.2*rng.normal(size=300)>0).astype(int)
m=make_pipeline(StandardScaler(),LogisticRegression()).fit(X[:220],y[:220])
r=permutation_importance(m,X[220:],y[220:],n_repeats=5,
    random_state=20,scoring='balanced_accuracy',n_jobs=1)
assert int(np.argmax(r.importances_mean))==0
RESULTADO={'importancia_media':r.importances_mean.round(3).tolist(),
    'dispersao':r.importances_std.round(3).tolist()}
print(RESULTADO)
```

Resultado de referência calculado

```
{"dispersao": [0.025, 0.006, 0.007], "importancia_media": [0.432, 0.007, 0.005]}
```

Leitura do código

O sinal de y vem de X0 com ruído pequeno; as outras duas colunas são auxiliares sem relação sistemática no mecanismo. `permutation_importance` usa somente teste e cinco repetições. O `argmax` esperado em zero é um teste de sanidade do mecanismo conhecido. A dispersão descreve variabilidade da permutação, não intervalo de causalidade.

As asserções verificam o contrato indicado no capítulo. O resultado acima foi produzido pela execução deste código. Valores em bytes e resultados de ponto flutuante podem variar conforme plataforma e versão; conserve as propriedades testadas e as tolerâncias explicitadas.

CAPÍTULO 12 · EXERCÍCIOS AUTORAIS

12.3 Caderno de aplicação

Exercício A · Modificação controlada

Duplique X0 com pequeno ruído e repita a permutação. Compare importância individual e permutação conjunta das duas colunas. Em seguida, avalie um teste cuja relação alvo-X0 foi invertida e registre a queda de desempenho.

Comentário de solução

A redundância permite que o modelo use a coluna não permutada, reduzindo a perda individual. Permutar ambas rompe o sinal compartilhado e mede uma contribuição conjunta diferente. Inverter a relação de geração altera o conceito, não apenas a distribuição de entradas; o pipeline antigo falha mesmo com valores de X aparentemente familiares. Recalibrar sem novos rótulos não corrige uma relação invertida.

Exercício B · Aplicação ampliada

Escreva uma ficha de modelo com finalidade, horizonte, esquema, blocos de treino, métricas, limiar, limites e procedimento de reavaliação. Acrescente teste automático que rejeite campos fora de ordem e categoria não prevista conforme a política definida.

Critérios específicos de revisão

Interprete importância como dependência preditiva, preserve o pipeline e monitore erros por território após qualquer transferência.

EVIDÊNCIA	REGISTRO A ENTREGAR
Experimento	Código original e modificação com sua hipótese.
Resultado	Saídas antes e depois; verificação numérica ou geométrica.
Interpretação	Explicação da diferença e limite de aplicação ao território.
Reprodução	Versões, parâmetros, seed e arquivo de entrada quando houver.

Responda antes de consultar o comentário. A aplicação ampliada é aberta e não possui um único número oficial: sua avaliação depende de cumprir os critérios específicos, conservar o contrato e justificar a hipótese escolhida.

Referências e ambiente reproduzível

Fontes técnicas primárias para consultar assinaturas de API, modelos de dados e hipóteses de uso. A organização didática, os exercícios e os dados sintéticos são próprios deste volume. Documentação de software não substitui avaliação do desenho do estudo.

- https://scikit-learn.org/stable/modules/cross_validation.html
- <https://scikit-learn.org/stable/modules/compose.html>
- https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- <https://scikit-learn.org/stable/modules/ensemble.html#forest>
- <https://scikit-learn.org/stable/modules/calibration.html>
- https://scikit-learn.org/stable/modules/permutation_importance.html

Ambiente que executou os laboratórios

Python: 3.13.5; geopandas: 1.2.0; shapely: 2.1.2; scikit-learn: 1.7.1; numpy: 2.2.5; pandas: 3.0.6; torch: 2.8.0+cpu; rasterio: 1.4.3; PyMuPDF: 1.28.2

Instalação e extração

```
python -m venv .venv
# Windows: .venv/Scripts/python.exe
python -m pip install -r requirements.txt
python capitulo_01.py
```

O requirements.txt está incorporado ao PDF e enumera as dependências do volume. A instalação deve ocorrer no ambiente virtual escolhido. Para PyTorch CPU, use o índice oficial de wheels CPU conforme a documentação da plataforma. Não é necessário baixar dados ou pesos para executar os laboratórios.

Os scripts não dependem de execução prévia de capítulos e os arquivos de saída de demonstração usam diretórios temporários. Para adaptar a dados reais, substitua entradas com contrato equivalente e revise datas, suporte e critérios; não apenas os nomes dos arquivos. O anexo resultados_laboratorios.json permite conferir as saídas da edição.

Projeto integrador e avaliação

Priorizar vistorias de unidades territoriais com regressão logística e floresta aleatória. Organize validação por blocos, escolha limiar segundo custo de falso negativo e orçamento, avalie o teste uma única vez e entregue pipeline, métricas por bloco e análise dos casos de erro.

Entregáveis

- Relatório com pergunta, unidade de análise, período, fontes e hipótese do procedimento.
- Código organizado em etapas, com contrato de entrada e asserções coerentes com os capítulos.
- Tabela de verificação que confronte resultado principal com uma alternativa ou cálculo de referência.
- Artefatos reabertos em processo novo, acompanhados de manifesto de versões e parâmetros.
- Discussão das limitações específicas e dos dados adicionais necessários para aplicação real.

DIMENSÃO	CRITÉRIO	PONTOS
Escopo e suporte	Pergunta e unidades explícitas, dados identificados.	15
Programação	APIs corretas, fluxo reproduzível e tratamento de casos inválidos.	25
Avaliação	Controles específicos, baseline ou conservação verificável.	25
Interpretação	Separar resultado numérico, hipótese e conclusão territorial.	20
Entrega	Arquivos, documentação e reprodução por outro estudante.	15

Revisão final

Escolha dois capítulos que tenham alterado uma decisão do projeto e explique como. Demonstre um caso em que o fluxo deve rejeitar a entrada e outro em que a saída é matematicamente válida, mas inadequada à pergunta. Essa distinção é necessária para concluir a entrega.

O avaliador deve conseguir localizar cada fonte, executar o código e conferir uma saída sem reconstruir escolhas não documentadas. Registre erros encontrados e correções realizadas. Uma métrica elevada ou um mapa bem apresentado não compensa uma referência espacial, data, máscara ou divisão de dados incompatível.